

Verification Plan

For the 8-bit Combinational ALU

Presented to:

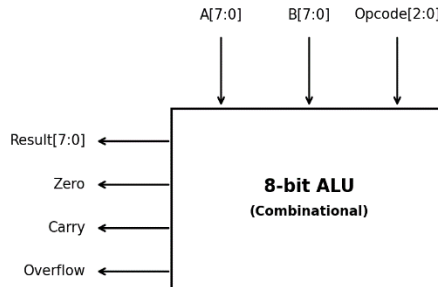
Analog Devices

Eng. Mohamed El Adawy

Presented by:

Mohamed Abdelhafez Elnewehy

Verification Plan For 8-bit ALU



- 8-bit ALU supporting 2 arithmetic, 2 logical and 2 shift operations
- Fully combinational design - no clock, no reset. Outputs update immediately when A, B or Opcode change
- Opcode selects the operation: "000"=ADD, "001"=SUB, "010"=AND, "011"=OR, "100"=SHL, "101"=SHR
- Opcodes "110" and "111" are invalid, Result is forced to 0
- Shift amount for SHL/SHR is taken from B[2:0], range 0 to 7
- "Carry" flag on ADD is the carry-out of the addition, on SUB it is the borrow flag (1 when A < B unsigned)
- "Overflow" flag active high on ADD/SUB whenever a signed (2's complement) overflow occurs
- "Zero" flag active high whenever the Result value is 0, for any opcode including invalid opcodes

Verification Plan

Signals

- 1] A , B → Valid 8-bit operands, cover min (0x00), max (0xFF), and mid-range values
- 2] Opcode → Combinational, selects the ALU operation immediately (no clock edge)
 - 000 → ADD : Result = A + B , Carry = carry-out , Overflow = signed overflow
 - 001 → SUB : Result = A - B , Carry = borrow flag , Overflow = signed overflow
 - 010 → AND : Result = A & B , Carry = 0 , Overflow = 0
 - 011 → OR : Result = A | B , Carry = 0 , Overflow = 0
 - 100 → SHL : Result = A << B[2:0] , Carry = 0 , Overflow = 0
 - 101 → SHR : Result = A >> B[2:0] , Carry = 0 , Overflow = 0
 - 110 / 111 → invalid opcode : Result forced to 0 , Carry = 0 , Overflow = 0
- 3] Result → Checker: Result matches the expected value of the selected opcode, updated as soon as inputs change
 - Checker: if opcode is invalid (110 or 111) → check Result == 0 regardless of A, B
 - Checker: shift amount uses only B[2:0] , upper bits of B must not affect SHL/SHR result
- 4] Carry → Checker: if opcode == ADD and (A+B) > 255 → Check Carry == 1 else == 0

- 5] Carry → Checker: if opcode == SUB and $A < B$ (unsigned) → Check Carry == 1 else == 0
- 6] Overflow → Checker: if opcode == ADD and A, B same sign but Result sign differs → Check Overflow == 1 else == 0
- 7] Overflow → Checker: if opcode == SUB and A, B different sign but Result sign differs from A → Check Overflow == 1 else == 0
- 8] Zero → Checker: if Result == 0 → Check Zero == 1 else == 0
 → Continuous check, since design is combinational (no pulse / no clock)

Functional Coverage

We need to cover all output signals:

Result	→ cover all opcodes
Result\ADD	→ sum overflows 8-bit range (255 + 1)
Result\SUB	→ difference underflows (0 - 1)
Carry	→ 1 if (ADD carry-out == 1)
Carry	→ 1 if (SUB borrow == 1, $A < B$)
Overflow	→ 1 if (ADD signed overflow, e.g. 127 + 1)
Overflow	→ 1 if (SUB signed overflow, e.g. -128 - 1)
Zero	→ 1 iff (Result == 0) for every opcode
Shift amount	→ cover shamt == 0 and shamt == 7 (min/max)
Opcode	→ cover all 8 encodings including invalid 110, 111
A, B	→ cover 0x00, 0xFF and random mid-range operands

Sequences & Constrained Randomization

1] ADD Sequence	→ with opcode == 000, random A, B
2] SUB Sequence	→ with opcode == 001, random A, B
3] AND Sequence	→ with opcode == 010, random A, B
4] OR Sequence	→ with opcode == 011, random A, B
5] SHL Sequence	→ with opcode == 100, random A, B[2:0]
6] SHR Sequence	→ with opcode == 101, random A, B[2:0]
7] Invalid Opcode Sequence	→ with opcode == 110 or opcode == 111
8] Corner Case Sequence	→ A , B constrained to {0x00 , 0x7F , 0x80 , 0xFF} to force carry/borrow/overflow

Test Scenarios

Scenario 1 : ADD Sequence for random number transaction range [5:10]

-- Goal : testing basic addition functionality

Scenario 2 : Corner Case Sequence → ADD with A=0xFF , B=0x01

-- Goal : testing unsigned carry-out wrap from 255 to 0

Scenario 3 : Corner Case Sequence → ADD with A=0x7F , B=0x01

-- Goal : testing signed overflow on addition (127 + 1)

Scenario 4 : Corner Case Sequence → SUB with A=0x00 , B=0x01

-- Goal : testing unsigned borrow / underflow wrap from 0 to 255

Scenario 5 : Corner Case Sequence → SUB with A=0x80 , B=0x01

-- Goal : testing signed overflow on subtraction (-128 - 1)

Scenario 6 : SHL Sequence → SHR Sequence with shamt = 0 then shamt = 7

-- Goal : testing shift boundaries and that upper bits of B are ignored

Scenario 7 : AND Sequence → OR Sequence → ADD → SUB → SHL → SHR ...etc.

-- Goal : Mixing the order of all operations back-to-back on consecutive input changes

Scenario 8 : Invalid Opcode Sequence → random for at least 10 times

-- Goal : testing that Result is forced to 0 and Zero == 1 for undefined opcodes

Test Cases (Applied and Verified)

All values below are decimal unless noted otherwise. These test cases were applied directly to the RTL and the outputs were confirmed to match the expected behavior for every corner case.

#	Opcode	A	B	Result	Carry	Overflow	Zero	Purpose
1	000 (ADD)	10	20	30	0	0	0	Basic addition
2	000 (ADD)	255	1	0	1	0	1	Unsigned carry-out wrap (255+1)
3	000 (ADD)	127	1	128	0	1	0	Signed overflow (127+1)
4	001 (SUB)	20	10	10	0	0	0	Basic subtraction
5	001 (SUB)	0	1	255	1	0	0	Unsigned borrow wrap (0-1)
6	001 (SUB)	128	1	127	0	1	0	Signed overflow (-128-1)
7	010 (AND)	240	15	0	0	0	1	Bitwise AND, zero result
8	011 (OR)	240	15	255	0	0	0	Bitwise OR, all bits set
9	100 (SHL)	1	0 (shamt=0)	1	0	0	0	Shift-left by 0 (no change)
10	100 (SHL)	1	7 (shamt=7)	128	0	0	0	Shift-left by max amount (7)
11	101 (SHR)	128	7 (shamt=7)	1	0	0	0	Shift-right by max amount (7)
12	010 (AND)	240	0	0	0	0	1	Zero flag via AND
13	110 (invalid)	170	85	0	0	0	1	Invalid opcode forces Result=0
14	111 (invalid)	170	85	0	0	0	1	Invalid opcode forces Result=0